Sparse Koopman Autoencoders Identify Local Dynamical Regimes in Multibasin Systems

Aidan Li, Uday Kiran Reddy Tadipatri, Mahan Fathi, Sarath Chandar, Ross Goroshin

Roadmap

1. **Motivation:** linearizing nonlinear dynamics
2. **Problem:** why multibasin systems break it
3. **Idea:** sparse supports as regime variables
4. **Model:** SKAE and its training
5. **Experiments:** forecasting, interventions, basins
6. **Discussion & outlook**

Motivation: Koopman learning

Objective

Embed nonlinear dynamics into coordinates where the evolution is **linear**, or at least locally linear.

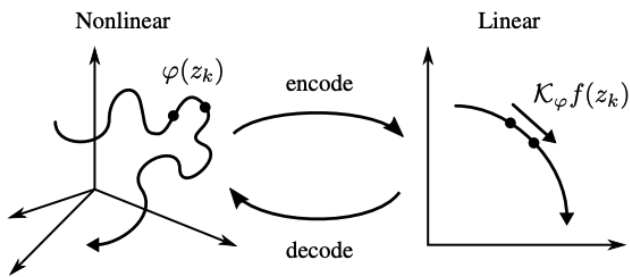


Figure from Azencot et al. (2020).

Motivation for linear coordinates

- Linear models make prediction, estimation, and control easier.
- A suitable change of coordinates can substantially simplify analysis.

Existence results for linearizing coordinates

- Local linearization near hyperbolic fixed points (Hartman–Grobman)
- Koopman eigenfunctions linearize on a basin of attraction...

but only *existentially*.

Good linear coordinates can exist, but how to build them from finite data?

A data-driven approach

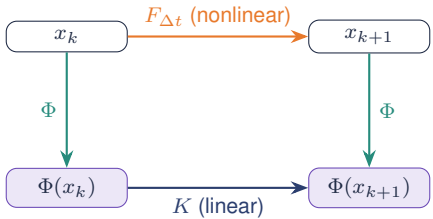
Rather than hand-designing observables,
learn the embedding directly from trajectory data.

DMD $\rightarrow$ eDMD $\rightarrow$ **Koopman autoencoders**

Koopman operator theory

The **Koopman operator** $\mathcal{K}$ acts *linearly* on scalar observables $g : \mathcal{X} \to \mathbb{R}$:

$$(\mathcal{K}g)(x) = g\Big(F_{\Delta t}(x)\Big)$$



Stack d_z such observables into a vector lift $\Phi = (g_1, \ldots, g_{d_z})$; a finite approximation seeks a matrix K with $\Phi(F_{\Delta t}(x)) \approx K\,\Phi(x)$, keeping enough to reconstruct x.

Koopman autoencoders (KAEs)

$$\boxed{\hat{x}_{k+h} \;=\; D_\phi\big(K^h\, E_\theta(x_k)\big)}$$

Learn encoder E_θ, decoder D_ϕ, and one latent transition K jointly, from trajectory windows.

A key modeling constraint

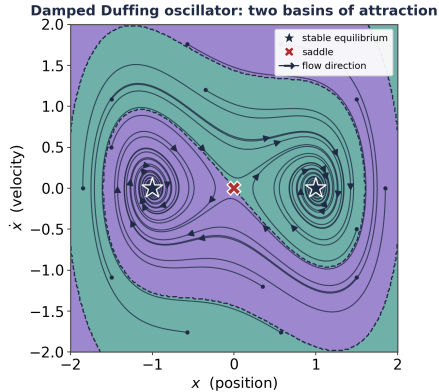
The same transition matrix K is applied at *every* rollout step.

Any state-dependent structure required for prediction must reside in the learned representation z_k.

Problem: multibasin systems

Many systems are multistable

Different initial conditions settle into **different** long-run behaviors.



No single global linearization

Multibasin systems *cannot generally* admit a single finite-dimensional *global* linearization with a continuous encoder–decoder pair.

Lan & Mezić 2013 · Brunton et al. 2016 · Fathi et al. 2024. · Pan & Duraisamy 2024 · Liu et al. 2025.

Reframing the objective

Don’t force all basins through a *single* global linearization.

Find distinct *local* linearizations.

Idea: sparse supports as regime variables

Two quantities the representation must encode

(i) **Which** local linearization is active

a *discrete* regime label

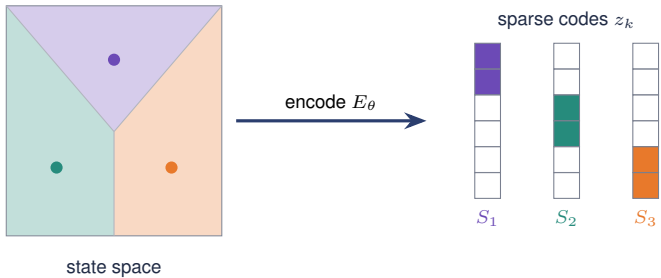
(ii) **Coordinates** that linearize dynamics within it

a *continuous* local chart

A sparse code encodes both quantities

the **support** (which coordinates are active) $\Rightarrow$ *which* regime

the **nonzero values** $\Rightarrow$ local Koopman coordinates



Each region of state space activates its own *support*: the colored (nonzero) cells; the code is zero elsewhere. Nearby states share a support, giving a union-of-subspaces structure.

Sparse coding

Given an overcomplete dictionary D, find a code with **few nonzeros**:

$$z^\star(x) = \arg\min_z \ \tfrac{1}{2}\|x - Dz\|_2^2 + \lambda_{\mathrm{sc}}\|z\|_1$$

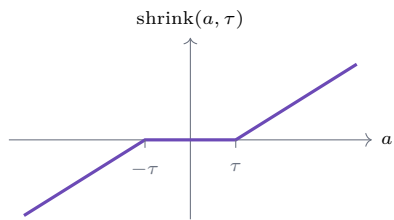
The ℓ_1 penalty drives most coordinates to *exactly* zero.

Learned ISTA (LISTA): sparse coding as an encoder

Unroll the solver into a fixed-depth network: from a dense pre-code $c = f_\theta(x)$, apply Q learned shrinkage steps.

$$u^{(0)} = \mathrm{shrink}(c, \tau), \quad u^{(q+1)} = \mathrm{shrink}\big(S\,u^{(q)} + c, \tau\big), \quad z = u^{(Q)} = E_\theta(x)$$

the $u^{(q)}$ are the shrinkage iterates; the sparse code is the final iterate $z = u^{(Q)}$.



Soft-thresholding zeros out a dead zone $[-\tau, \tau]$, creating the sparse support.

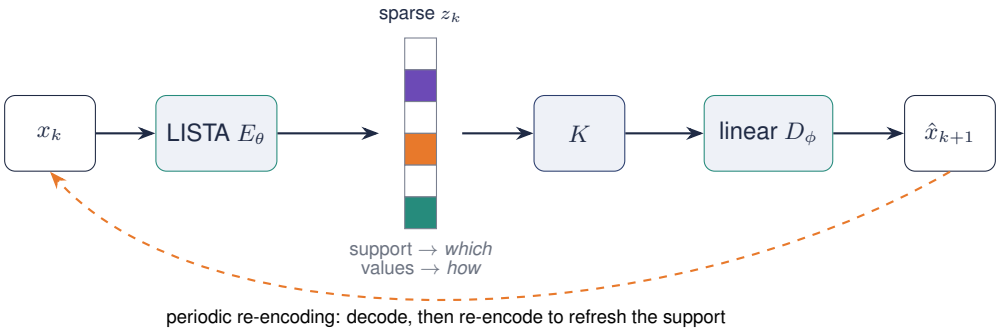
The relevant inductive bias

Thresholding makes *exact zeros*.

The encoder **explicitly selects** active coordinates,
and *learns the selection rule* from the prediction objective.

The SKAE model & training

Sparse Koopman Autoencoder (SKAE)



A LISTA encoder produces a sparse code; a unit-norm linear dictionary decodes; one matrix K advances the latent.

Two complementary objectives

- **Koopman:** values on each support evolve *linearly* under K
- **Sparsity:** different regimes use *different* active coordinates

Jointly, the discrete support selects the regime while the continuous values parameterize its local linear dynamics.

Encoders compared

- **LISTA:** sparse by construction (thresholding gives exact zeros)
- **Sparse MLP:** induced sparsity (ReLU with an ℓ_1 penalty)
- **Dense MLP:** no sparsity mechanism (control baseline)

Training objective: four terms

- **Prediction:** decoded rollout matches future states
- **Reconstruction:** code stays tied to the observed state
- **Latent linearity:** rolled-out latent matches encoded latent
- **Sparsity:** few active coordinates

$$\mathcal{L} = \lambda_{\text{pred}}\bar{\mathcal{L}}_{\text{pred}} + \lambda_{\text{rec}}\bar{\mathcal{L}}_{\text{rec}} + \lambda_{\text{lin}}\bar{\mathcal{L}}_{\text{lin}} + \lambda_{\text{sp}}\bar{\mathcal{L}}_{\text{sp}} + \mathcal{L}_{\text{struct}}$$

No basin labels enter training.

Support objects

Exact support: $S_{\mathrm{abs}}(x) = \{\, i : |z_i| > 10^{-3} \,\}$, the set of active coordinates.

Support families group nearby exact supports by *Jaccard overlap* (intersection over union), merging when $J \geq 0.5$:

$$J(m, r) = \frac{|\,\mathrm{active}(m) \cap \mathrm{active}(r)\,|}{|\,\mathrm{active}(m) \cup \mathrm{active}(r)\,|}$$

We use these for interventional studies on whether the coordinates are used and how support families track basins.

Periodic re-encoding refreshes the support

Every m steps the predicted latent is decoded and re-encoded, so the encoder re-selects the active support as the trajectory advances.

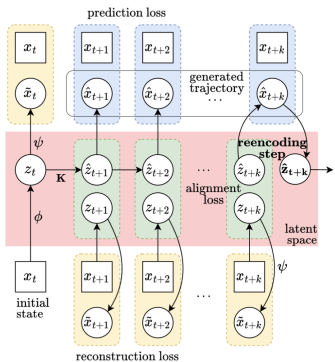


Figure from Fathi et al., ICLR 2024. Squares = ground truth, circles = model-inferred; ϕ, ψ are the encoder/decoder (our E_θ, D_ϕ).

Experiments

Benchmarks: 25 systems

- **15 procedural 2-D multibasin systems**

  Gaussian wells + rotational drift; *known* basin geometry (eval only)

- **10 chaotic Dysts flows**

  Chua, Shimizu–Morioka, and more: more challenging flows

Six models compared

- **Sparse LISTA encoders**

  LISTA · LISTA-BD (block-diag. K) · LISTA-SB (soft-block K)

- **Sparse MLP encoders** ReLU $+ \ell_1$

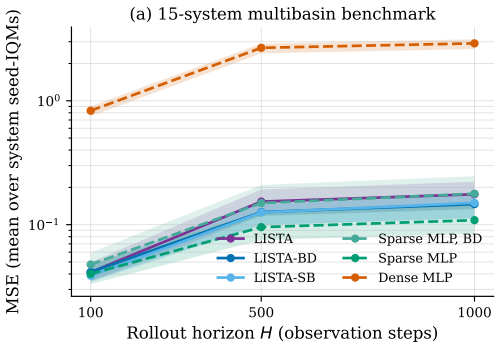
  Sparse MLP · Sparse MLP-BD (block-diag. K)

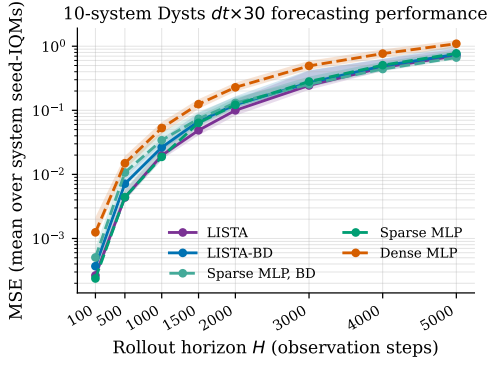
- **Dense MLP** no sparsity: the control baseline

Experimental questions

1. Do sparse encoders lead to better *forecasting*?
2. Are the supports *functionally necessary*?
3. Do support families *align with basins*?

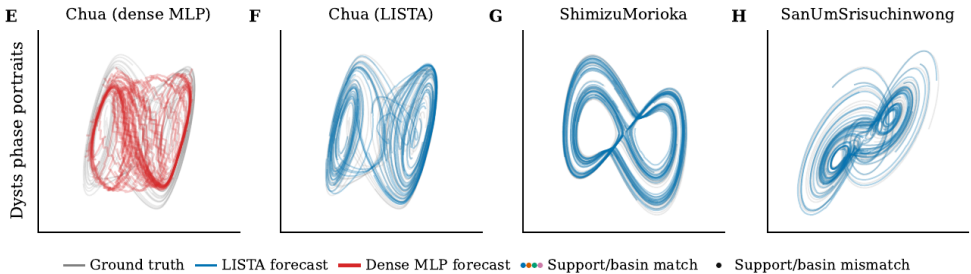
Result 1: sparse encoders forecast better





Sparse models improve on the dense baseline at *every* horizon; at $H = 1000$ the dense baseline incurs **17–27**$\times$ higher error.

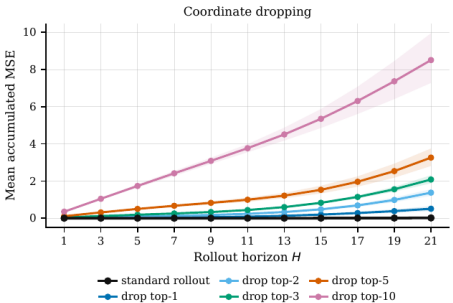
Result 1: qualitative behavior on chaotic flows

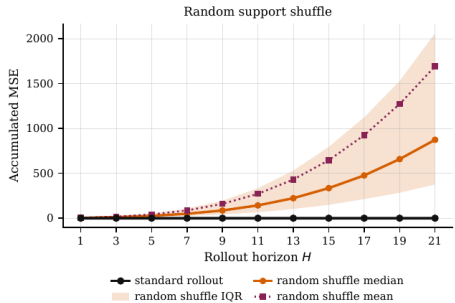


The **dense MLP (red)** forecasting is much qualitatively worse than the **LISTA (blue)** forecast.
Sparsity improves long-horizon forecasts.

Result 2: are the supports functionally necessary?

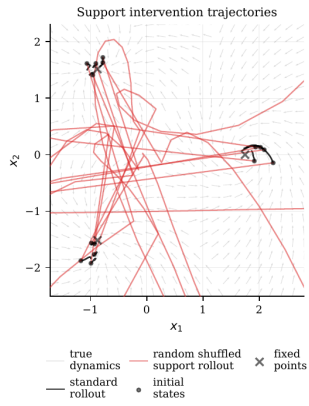
Corrupt only the *initial active set*, keep the values fixed, then forecast.





Zeroing the top-k active coefficients sharply increases error even at short horizons (left); random reassignment to inactive coordinates is substantially more damaging (right).

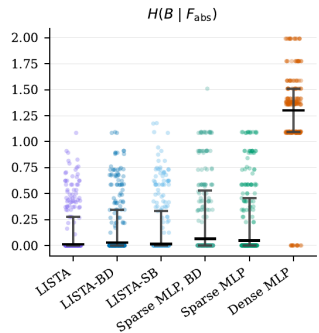
Result 2: trajectory-level effect



Randomly reassigning the active coordinates (red) drives trajectories away from the true dynamics; standard rollouts (black) remain accurate.

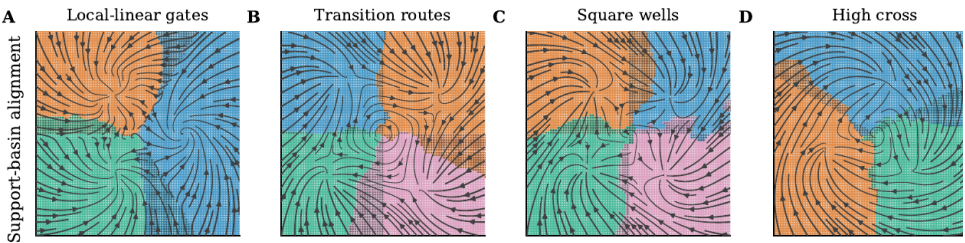
Result 3: support families identify the basin

How much does a support family reveal about the withheld label?



LISTA reaches the lowest conditional entropy (0.130); **dense MLP** sits near maximal uncertainty.

Result 3: unsupervised basin recovery



LISTA support families partition the flows; coloured = support/basin *matches*. LISTA exposes ≈ 4.0 families vs. the benchmark mean/median 4.20/4; **dense MLP** collapses to 1. No basin labels are used during training.

Discussion & outlook

A practical reconciliation

Theory: no single global finite Koopman embedding for multibasin systems.

Practice: a single sparse model *can* hold many local Koopman regimes, indexed by latent support.

Interpretability from the code structure

Sparse supports are an *interface* between
continuous Koopman coordinates and discrete regimes:

a simple object you can inspect after training.

(WIP) Future work: toward structured world models

World models (Dreamer, V-JEPA 2, Genie) predict in *latent space*, but their latent dynamics are largely *unstructured*.

- **Supports as regime latents:** rollouts that switch local linear laws *and report which law is active*; a label-free, amortized switching system
- **Control:** action-conditioned SKAE with per-regime linear MPC / LQR in the lifted space
- **Robustness & scale:** noise, partial observability, pixels; uncertainty-aware Koopman operators under distribution shift

Hafner et al. 2023–25 (Dreamer) · Assran et al. 2025 (V-JEPA 2) · Rozwood et al. 2024 (Koopman-assisted RL) · Linderman et al. 2017 (SLDS)

Conclusion

Interpretability in dynamical representation learning can come from the **structure of the latent code**.

Backup slides

Backup: statistical protocol

- **Point estimates:** interquartile mean (IQM) over seeds within each system, then mean across systems.
- **Paired effect (MSE):** $\Delta = \log_{10}(E_{\text{cand}}/E_{\text{Dense}})$, one-sided alternative $\Delta < 0$.
- **Multibasin & support diagnostics:** unit = seed; within-system Wilcoxon; Holm across systems.
- **Dysts:** unit = system; exact one-sided sign test; a superscript means improvement on all 10/10.
- **Intervention panels:** descriptive mechanism diagnostics (bootstrap / IQR bands), not tests.

Backup: training details

Controlled multibasin

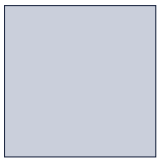
- rollout $L = 8$, 200k steps, batch 256, AdamW
- lr: enc/dec 5×10^{-5}, transition 5×10^{-6}
- $\lambda_{\text{pred}} = 1,\ \lambda_{\text{rec}} = 0.03,\ \lambda_{\text{lin}} = 1,\ \lambda_{\text{sp}} = 3 \times 10^{-3}$
- LISTA: threshold $\alpha = 0.15$, 2 refinement loops
- label-free boundary-emphasized resets

Dysts $dt \times 30$

- rollout $L = 10$, 100k steps, batch 256
- $\lambda_{\text{sp}} = 6 \times 10^{-3}$ (dense: 0)
- deterministic caches; disjoint train/val/test
- best re-encoding period $m \in \{10, \ldots, 200\}$
- $d_z = 256$ latent dimension throughout

Backup: three structures for the transition K

Dense

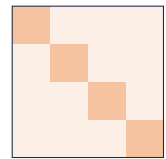


all coordinates mix

Block-diagonal

M groups, no cross-talk

Soft-block



off-block entries penalized

Varied *independently* of the encoder, to probe how block structure interacts with learned supports.

Backup: two evaluation diagnostics

- $H(B \mid F_{\text{abs}})$: uncertainty about the basin given the family

  lower is better

- $|F_{\text{abs}}|$: number of families

  compare to the number of basins

Basin labels are used *only* here, after training, never to train or select the model.